Shell Data Platform: DevOps Architecture and Access Requests

New parallel release process for ADF, Databricks and the Azure SQL config DB · GitHub Actions + Azure Boards · Prepared by Vamshi Singam · 23 October 2026

What this document is for

- Section 1–4: the target architecture, so admins understand *why* each permission is needed.
- Section 5: **every access request, grouped by the team that grants it** (47 items). The attached Excel tracker has the same list with status columns for admins to fill in.
- Section 6: open questions that change some requests. Please answer these first.

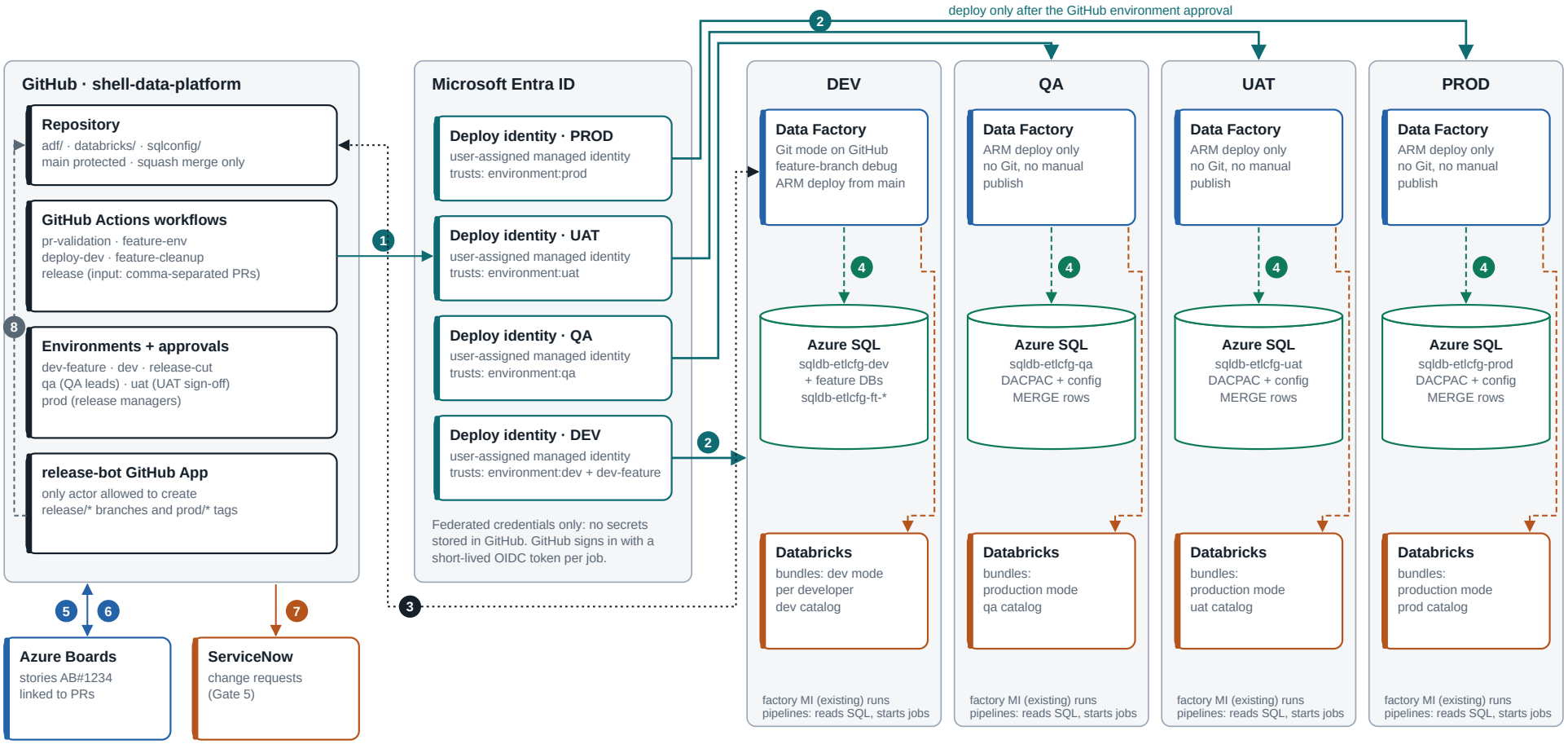
Principles: every developer's work is isolated and fully testable on a feature branch; only approved PRs reach `main`; releases move a chosen list of PRs; GitHub holds **no Azure secrets** (federated credentials on the existing managed identities).

1. Scope

Project type	Source folder	Built as	Deployed with	Feature-branch testing
Azure Data Factory	`adf/`	ARM template (ADF utilities npm package)	ARM deployment + pre/post trigger script	DEV factory in Git mode: debug runs use the branch
Databricks	`databricks/`	Databricks Asset Bundle	`databricks bundle deploy -t <env>`	Bundle development mode: jobs prefixed per developer, personal `dev_<user>` schema
Azure SQL config DB	`sqlconfig/`	SQL project → DACPAC + one MERGE file per config key	`sqlpackage Publish` with SQLCMD variables per env	Own database per branch: `sqldb-etlcfg-ft-<story>` (copy of DEV + branch scripts)

Environments: DEV, QA, UAT, PROD, each with its own factory, config database and Databricks workspace. Stories live in Azure Boards; PRs link with `AB#<id>`. Production changes need an approved ServiceNow CR.

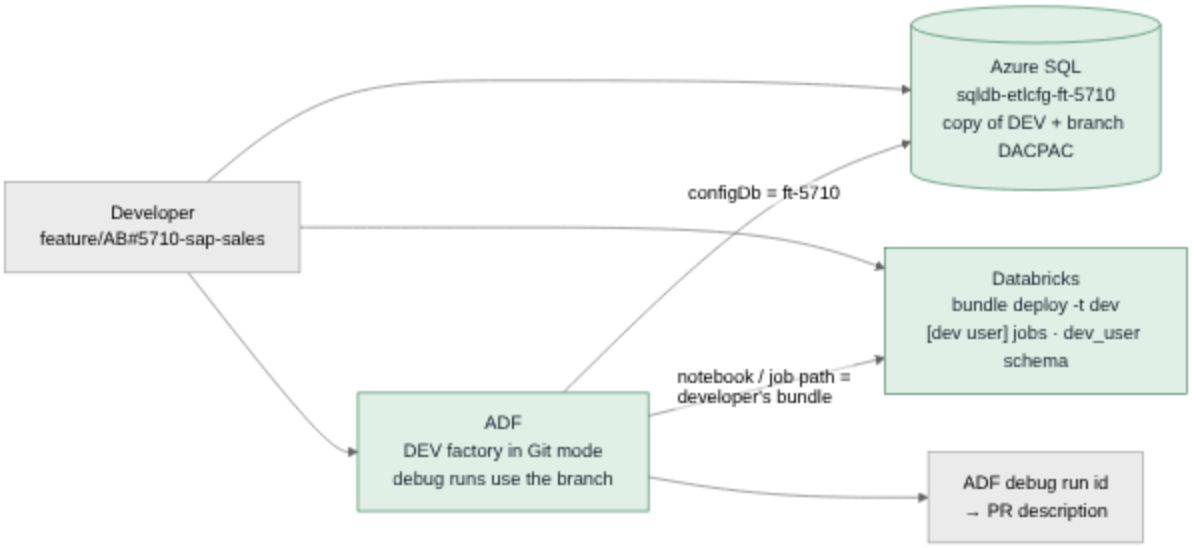
2. Architecture and service-to-service authentication



How each connection authenticates

1. GitHub workflow → Entra ID: OIDC token exchanged for the environment's managed identity
2. Managed identity → Azure: ARM (ADF), Entra login to SQL (DACPAC), Databricks CLI as workspace service principal
3. ADF Studio (DEV) and Databricks Git folders ↔ GitHub: org-approved GitHub apps, per-developer sign-in
4. Factory managed identity (existing) → config DB (db reader/writer) and Databricks (CAN MANAGE RUN)
5. Azure Boards app ↔ GitHub: AB#1234 in a PR links the story
6. Release workflow → Azure Boards API: managed identity added to the Azure DevOps org
7. Release workflow → ServiceNow Table API: integration user, secret held in the 'prod' environment
8. release-bot GitHub App token → repo: cuts release/<id>, pushes prod/<id> tags

3. Developer isolation: test everything before the PR



1. Developer creates `feature/AB#<story>-…` and works on ADF, Databricks and SQL config together.
2. On push, **feature-env** creates `sqldb-etlcfg-ft-<story>` (copy of the DEV config DB, which always equals `main`) and deploys the branch's DACPAC and config rows to it.
3. The developer deploys the bundle in development mode (`databricks bundle deploy -t dev`), which creates their own prefixed copies of jobs.
4. In ADF Studio (Git mode, branch selected) the developer debugs the pipeline with parameters `configDb = sqldb-etlcfg-ft-<story>` and the job/notebook path of their bundle. The shared DEV database and other developers are untouched.
5. The PR includes `AB#<story>` and `Feature-Test-Run: <ADF run id>`. After merge, **deploy-dev** updates DEV and **feature-cleanup** drops the feature DB.

4. GitHub workflows and the identity each uses

Workflow	Trigger	GitHub environment	Azure identity	What it does
pr-validation	pull_request to main	none	CI identity (pull_request credential)	AB# story check, ADF validate, bundle validate, DACPAC build, Feature-Test-Run check
feature-env	push to feature/**, bugfix/**	dev-feature	DEV deploy identity	Copy DEV config DB → feature DB, deploy branch DACPAC + config, grant developers write on it
feature-cleanup	PR closed + nightly	dev-feature	DEV deploy identity	Drop feature DBs whose branch is merged, closed or older than 14 days
deploy-dev	push to main	dev	DEV deploy identity	Deploy config DB → Databricks bundle → ADF to DEV (integration)

release	manual: Run workflow with PR list	release-cut, qa, uat, prod	QA / UAT / PROD deploy identities; release-bot app	Cut release/<id> from the listed PRs, build once, deploy QA → UAT, check ServiceNow CR, deploy PROD, tag, update stories and CR



Run workflow: release (GitHub → Actions)

PRs to release (comma-separated)

`1234,1240,1256`

Release id

`R2026.11.01`

ServiceNow CR (or "auto")

`auto`

Run workflow

Same model as the Marathon Azure DevOps release: only the listed PRs move. Each environment job then waits for its approvers.

5. Access requests by owner

Granted by	Items	Needed to start (P1)
GitHub org owner	12	9
Entra ID admin / identity owner	6	4
Azure subscription owner (RBAC)	8	7
Azure SQL server Entra admin	4	4
Databricks account admin	1	1
Databricks workspace admin / UC catalog owner	8	7
Azure DevOps (Boards) project admin	2	1
ServiceNow platform admin	2	0
Network / cloud platform team	2	1
Existing data/DevOps team (information)	2	1
Total	**47**	**35**

Phases: P1 Start building: DEV, feature-branch testing, PR checks · **P2** First release to QA and UAT · **P3** First release to PROD.

Two checks before anything else: (1) the managed identities we reuse must be **user-assigned**. Azure does not allow federated credentials on system-assigned identities, so GitHub could not use them. (2) Required reviewers on GitHub Environments for private/internal repos need **GitHub Enterprise**. Without them, Gates 3, 4 and 6 have no approval step.

GitHub org owner 12

ID	Principal / target	Env	Permission / action	Why	Phase
GH-01	Vamshi Singam (lead)	—	Create private/internal repo <shell-org>/shell-data-platform; give the lead Admin role on it during setup	New parallel process needs its own repo with ADF, Databricks and SQL projects side by side	P1
GH-02	GitHub teams: developers, tech-leads, release-managers, qa-leads, uat-signoff	—	Create or map teams (ideally synced from Entra groups); repo roles: developers = Write, tech-leads = Maintain, release-managers = Maintain	Drives CODEOWNERS review and environment approvers	P1
GH-03	Repo settings	—	Allow squash merge only (disable merge commits and rebase merge); auto-delete head branches	One PR = one commit on main, which makes releases selectable by PR number	P1
GH-04	Ruleset on main	—	Require PR; 1 approval from CODEOWNERS (tech-leads); dismiss stale approvals; require status check 'pr-validation'; require conversation resolution; block force-push and deletion; no bypass except org admins	Gate 1: only reviewed, tested PRs reach main	P1
GH-05	Ruleset on release/* branches and prod/* tags	—	Restrict create/update/delete; bypass list = release-bot GitHub App only	Nobody hand-edits a release branch or moves a production tag	P2
GH-06	GitHub App 'shell-data-release-bot'	—	Create app (Contents: read & write, Pull requests: read, Metadata: read), install on the repo only; give its App ID and private key to the lead for the 'release-cut' environment secret	Cuts release/<id> branches and pushes prod/<id> tags past the ruleset in GH-05	P2
GH-07	GitHub Actions policy	—	Enable Actions on the repo; allow actions: actions/*, azure/login, databricks/setup-cli, Azure/arm-deploy, Azure/sql-action (or allow 'verified creators')	Workflows for validation, feature environments and releases	P1
GH-08	Workflow permissions / OIDC	—	Default GITHUB_TOKEN = read; workflows may request id-token: write (OIDC). Confirm no org policy blocks OIDC	Passwordless sign-in to Azure with the existing managed identities	P1
GH-09	GitHub Environments	DEV · QA · UAT · PROD	Create environments: dev-feature, dev, release-cut, qa, uat, prod. Required reviewers: qa = qa-leads, uat = uat-signoff, prod = release-managers. Deployment branches: dev-feature = feature/**, bugfix/**; dev = main; release-cut = main; qa/uat/prod = release/* Required reviewers on a private/internal repo need GitHub Enterprise (confirm plan, see OQ-07)	Gates 3, 4 and 6 are environment approvals	P1
GH-10	Environment secrets	UAT · PROD	Store ServiceNow integration credentials as secrets in the 'prod' environment only (and 'uat' if the CR is checked before UAT)	Gate 5 reads and updates the change request	P3

ID	Principal / target	Env	Permission / action	Why	Phase
GH-11	Third-party app access	DEV	Approve the 'Azure Data Factory' and 'Databricks' GitHub apps / OAuth apps for the org (if OAuth app access restrictions are on) Blocking for feature testing	DEV factory Git mode and Databricks Git folders must read feature branches: this is what makes feature-branch testing possible	P1
GH-12	GitHub Actions runners	DEV · QA · UAT · PROD	GitHub-hosted runners if Azure endpoints are public. If ADF/SQL/Databricks are private: self-hosted runners in a VNet with line of sight (or GitHub-hosted runners with Azure private networking), registered to this repo Depends on OQ-03	Runners must reach SQL (DACPAC), Databricks and ARM endpoints	P1

Entra ID admin / identity owner 6

ID	Principal / target	Env	Permission / action	Why	Phase
ID-01	Existing managed identities	DEV · QA · UAT · PROD	Confirm each identity we reuse is USER-assigned and give: name, client ID, object ID, resource group, tenant ID If they are system-assigned, request one user-assigned identity per environment instead (see OQ-04)	GitHub can only sign in through a user-assigned identity (or app registration); system-assigned identities (for example the factory's own) cannot take federated credentials	P1
ID-02	Deploy identity – DEV	DEV	Add federated credentials: issuer https://token.actions.githubusercontent.com, audience api://AzureADTokenExchange, subjects repo:<shell-org>/shell-data-platform:environment:dev and ...:environment:dev-feature Needs Contributor/Owner on the identity to add	deploy-dev and feature-env workflows	P1
ID-03	CI identity (read-only) – DEV	DEV	Federated credential subject repo:<shell-org>/shell-data-platform:pull_request (a separate low-privilege identity is preferred; DEV deploy identity acceptable)	pr-validation runs ADF validate and bundle validate against DEV	P1
ID-04	Deploy identity – QA, UAT	QA · UAT	Federated credentials with subjects ...:environment:qa and ...:environment:uat (on each environment's identity)	release workflow QA and UAT jobs	P2
ID-05	Deploy identity – PROD	PROD	Federated credential with subject ...:environment:prod	release workflow PROD job (runs only after release-manager approval)	P3
ID-06	Entra security groups	—	Create or confirm groups: grp-data-developers, grp-data-techleads, grp-data-qa, grp-data-uat-signoff, grp-data-release-managers	Used for Azure RBAC, SQL, Databricks and GitHub team sync	P1

Azure subscription owner (RBAC) 8

ID	Principal / target	Env	Permission / action	Why	Phase
AZ-01	Deploy identity – each env	DEV · QA · UAT · PROD	Data Factory Contributor on the resource group that holds that environment's factory	ARM template deployment, stop/start triggers, delete removed pipelines	P1 DEV · P2 QA/UAT · P3 PROD
AZ-02	Deploy identity – DEV	DEV	SQL DB Contributor on the DEV SQL logical server DEV only	Create, copy and drop feature databases sqldb-etlcfg-ft-<story>	P1
AZ-03	DEV SQL logical server	DEV	Create elastic pool 'pool-etlcfg-feature' (small, e.g. GP 2 vCore or 50–100 eDTU) on the DEV server; name a cost owner Sizing/cost decision (OQ-05)	Feature databases share one capped pool; nightly cleanup keeps it small	P1
AZ-04	Deploy identity – each env	DEV · QA · UAT · PROD	Reader on the environment's data-platform resource group(s)	Look up resource IDs, run ARM what-if	P1 DEV · P2 QA/UAT · P3 PROD
AZ-05	grp-data-developers	DEV	Data Factory Contributor on the DEV factory only No write access to QA/UAT/PROD factories	Author in Git mode and debug pipelines on feature branches	P1
AZ-06	grp-data-developers	QA · UAT · PROD	Reader (or Data Factory monitoring via Reader) on QA/UAT/PROD factories	Monitor runs without changing anything	P2
AZ-07	Factory managed identity (existing) – each env	DEV · QA · UAT · PROD	Verify existing: Storage Blob Data Contributor on ADLS; Key Vault Secrets User on the Key Vault used by linked services Verification only	Pipelines keep working after deployments	P1
ADF-01	DEV factory	DEV	Connect the DEV factory to GitHub: repo shell-data-platform, collaboration branch main, root folder /adf; turn ON 'Disable publish' (automated publish) Done by the lead once AZ-05 and GH-11 are granted	Feature branches can be debugged; main is deployed by the pipeline, not the Publish button	P1

Azure SQL server Entra admin 4

ID	Principal / target	Env	Permission / action	Why	Phase
SQL-01	Each config SQL logical server	DEV · QA · UAT · PROD	Confirm Microsoft Entra admin is set and Entra authentication is enabled; give server and database names	Identities sign in without passwords	P1
SQL-02	Deploy identity – each env	DEV · QA · UAT · PROD	In the config DB: CREATE USER [<deploy-identity>] FROM EXTERNAL PROVIDER; ALTER ROLE db_owner ADD MEMBER [<deploy-identity>] In DEV this user is copied into every feature DB automatically	DACPAC deploys schema, permissions and config rows	P1 DEV · P2 QA/UAT · P3 PROD
SQL-03	Factory managed identity (existing) – each env	DEV · QA · UAT · PROD	Verify it is a user in the config DB with db_datareader, db_datawriter and EXECUTE Verification only	ADF reads config and writes watermarks; copied into feature DBs automatically	P1

ID	Principal / target	Env	Permission / action	Why	Phase
SQL-04	grp-data-developers	DEV	db_datareader on sqldb-etlcfg-dev	Inspect config; the feature-env workflow grants write access on each feature DB itself	P1

Databricks account admin 1

ID	Principal / target	Env	Permission / action	Why	Phase
DBX-01	Deploy identity – each env	DEV · QA · UAT · PROD	Add the identity (by client ID) as a service principal in the Databricks account; assign it to that environment's workspace with 'Workspace access'	Databricks CLI deploys asset bundles as this identity	P1 DEV · P2 QA/UAT · P3 PROD

Databricks workspace admin / UC catalog owner 8

ID	Principal / target	Env	Permission / action	Why	Phase
DBX-02	Deploy identity – each env	DEV · QA · UAT · PROD	CAN MANAGE on the bundle root folder (e.g. /Workspace/Shared/shell-data); allowed to create jobs	bundle deploy creates and updates jobs, notebooks and pipelines	P1 DEV · P2 QA/UAT · P3 PROD
DBX-03	Deploy identity – each env	DEV · QA · UAT · PROD	CAN USE on the job cluster policy, or serverless jobs enabled Depends on OQ-02	Jobs defined in the bundle can start compute	P1 DEV · P2 QA/UAT · P3 PROD
DBX-04	Deploy identity – each env	DEV · QA · UAT · PROD	If jobs run as an existing run-as service principal: 'Service Principal: User' role on that principal	Bundle sets run_as on production jobs	P2
DBX-05	Deploy identity – each env	DEV · QA · UAT · PROD	Unity Catalog on the environment catalog: USE CATALOG, USE SCHEMA, CREATE SCHEMA, CREATE TABLE, CREATE FUNCTION, CREATE VOLUME, MODIFY, SELECT (or ownership of bundle-managed schemas)	Bundle-managed DDL and grants	P1 DEV · P2 QA/UAT · P3 PROD
DBX-06	Deploy identity – each env	DEV · QA · UAT · PROD	CAN USE on one SQL warehouse	Run UC SQL steps during deployment	P1 DEV · P2 QA/UAT · P3 PROD
DBX-07	Factory managed identity (existing) – each env	DEV · QA · UAT · PROD	Verify it is a workspace service principal; give its client ID so the bundle grants it CAN MANAGE RUN on the jobs it triggers Verification + information	ADF triggers Databricks jobs after every deployment	P1
DBX-08	grp-data-developers	DEV	DEV workspace access; serverless/cluster use; on dev catalog: USE CATALOG, CREATE SCHEMA (personal dev_<user> schemas)	Developers deploy bundles in development mode: resources prefixed with their name, isolated per developer	P1
DBX-09	DEV workspace – Git integration	DEV	Allow Git folders to the GitHub org (add repo to the Git allowlist if restricted)	Developers work on feature branches inside the workspace	P1

Azure DevOps (Boards) project admin 2

ID	Principal / target	Env	Permission / action	Why	Phase
ADO-01	Azure Boards app	—	Install the Azure Boards app on the GitHub org and connect the ADO project to shell-data-platform (needs ADO project admin + GitHub org owner)	AB#1234 in a PR links it to the story; PR check blocks merges without a story	P1
ADO-02	Deploy identity (QA/UAT/PROD, or one release identity)	QA · UAT · PROD	Add the managed identity to the Azure DevOps organization (Basic access); Contributor on the project or area path	Release workflow moves stories to Ready for QA / Ready for UAT / Closed and comments the release id	P2

ServiceNow platform admin 2

ID	Principal / target	Env	Permission / action	Why	Phase
SN-01	Integration account / OAuth client	PROD	Integration user (or OAuth client credentials) able to read change_request and update state and work_notes for this team's assignment group	Gate 5: CR approved + in window + PR list matches; evidence written back after deploy	P3
SN-02	Change template	PROD	Add a field (e.g. u_release_prs) or agree the description lines 'Release-Id:' and 'Release-PRs:' on the CR template; confirm change model (Normal/Standard)	Ties each CR to the exact PRs being deployed	P3

Network / cloud platform team 2

ID	Principal / target	Env	Permission / action	Why	Phase
SQL-05	Config SQL servers – network	DEV · QA · UAT · PROD	Allow the runners from GH-12 to reach port 1433: private endpoint + VNet runners, or firewall rule for static runner IPs Depends on OQ-03	DACPAC deployment and feature-DB setup	P1
SN-03	ServiceNow instance – network	PROD	If the instance restricts inbound IPs, allow the runner egress IPs Conditional	Runners call the ServiceNow Table API	P3

Existing data/DevOps team (information) 2

ID	Principal / target	Env	Permission / action	Why	Phase
ADF-02	Integration runtimes	DEV · QA · UAT · PROD	Names/types of integration runtimes (Azure, self-hosted, shared/linked) used by each factory Information	ARM parameter files map linked services to the right runtime per environment	P1
ADO-03	Board process	—	Confirm work-item state names and the area path used by this team Information	Workflow uses the exact state names	P2

6. Open questions

ID	Question	What it affects	Who can answer
OQ-01	Is each environment (DEV, QA, UAT, PROD) in its own subscription? Resource group and resource names per env?	Scopes for every Azure role assignment	Vamshi / platform team
OQ-02	Is Databricks serverless (jobs and SQL) allowed, or must jobs use a cluster policy?	DBX-03 and bundle job definitions	Databricks admin
OQ-03	Are ADF, Azure SQL and Databricks behind private endpoints? Which self-hosted integration runtimes exist?	Runner choice (GH-12), SQL-05, ADF-02	Network team
OQ-04	Are the managed identities we must reuse user-assigned or system-assigned?	If system-assigned, GitHub cannot use them: new user-assigned identities are needed	Entra admin
OQ-05	Elastic pool size and cost owner for feature databases	AZ-03	Budget owner
OQ-06	How are Azure SQL scripts deployed today (SQL project/DACPAC, numbered scripts, other)?	Decides the SQL project tooling; process stays the same	Existing team
OQ-07	Which GitHub plan does the org have (Enterprise Cloud?)	Environment required reviewers on private/internal repos need GitHub Enterprise	GitHub org owner
OQ-08	ServiceNow: assignment group, change model, and whether CRs can be read/updated via API	SN-01, SN-02	ServiceNow admin

7. Implementation order

1. **P1, week 1–2:** repo, rulesets, environments, DEV identity + federated credentials, DEV RBAC/SQL/Databricks grants, ADF Git connection, Azure Boards app. Build pr-validation, feature-env, feature-cleanup, deploy-dev. Pilot one story end-to-end on a feature branch.
2. **P2, week 3:** QA/UAT identities and grants, release-bot app, release workflow through UAT, Azure Boards state updates.
3. **P3, week 4:** PROD identity and grants, ServiceNow integration, first PROD release under a CR. Then retire the old path for this team.